Strings (Basic & Medium) — Master Revision Sheet (Step 5, P01–P15)

> **One-stop revision.** Each entry = **Problem** (what's asked + tiny example) + **Recall** (the trigger line) + **Algorithm** + **Trap / don't-miss** + **Time / Space** + **Pattern**. Go top-to-bottom the night before an interview; every string problem in this step is here.

60-Second Index

#	Problem	LC	Pattern	Time / Space
P01	Remove Outermost Parentheses	1021	Depth counter	O(N) / O(N)
P02	Reverse Words in a String	151	Right-to-left scan	O(N) / O(N)
P03	Largest Odd Number in String	1903	Rightmost odd digit	O(N) / O(1)
P04	Longest Common Prefix	14	Sort, compare ends	O(N·M·log N) / O(1)
P05	Isomorphic Strings	205	Last-seen fingerprint	O(N) / O(1)
P06	Rotate String	796	Substring of s+s	O(N) / O(N)
P07	Check Anagram	242	Frequency count	O(N) / O(1)
P08	Sort Characters by Frequency	451*	Count + sort pairs	O(N) / O(1)
P09	Max Nesting Depth Parentheses	1614	Running counter	O(N) / O(1)
P10	Roman to Integer	13	Compare with next	O(N) / O(1)
P11	String to Integer (atoi)	8	Phased parse + clamp	O(N) / O(1)
P12	Count Substrings (exactly k distinct)	992*	atMost(k)−atMost(k−1)	O(N) / O(k)
P13	Longest Palindromic Substring	5	Expand around center	O(N²) / O(1)
P14	Sum of Beauty of All Substrings	1781	Incremental frequency	O(N²·26) / O(1)
P15	Reverse Every Word (= P02)	151	Right-to-left scan	O(N) / O(N)

* variant of the referenced LC problem

Pattern Toolbox (the shapes that repeat)

- **Frequency array / map** (P05, P07, P08, P12, P14): 26-array for letters (O(1) space) or hash map for general characters. `++` one side, `--` the other for comparison (anagram).
- **Depth / running counter** (P01, P09): brackets need only an integer, not a stack, when the input is guaranteed valid.
- **Two-pointer word parsing** (P02, P15): scan from the right, skip spaces, capture word by `[start, end]` bounds, append with a single separator.
- **Greedy on digits** (P03): oddness depends only on the last digit → extend as far right as possible.
- **Expand around center** (P13): every palindrome has a center; try all `2n-1` odd/even centers.
- **Sliding window + "atMost" trick** (P12): `exactly(k) = atMost(k) − atMost(k−1)`; window adds `right−left+1`.
- **Phased parsing / simulation** (P10, P11): process the string in ordered phases (spaces → sign → digits), clamp overflow with a wider type.
- **Doubling trick** (P06): a rotation of `s` is any substring of `s + s`.

P01 — Remove Outermost Parentheses

- **Problem:** Drop the outer pair of each primitive. `"(()())(())"` → `"()()()"`.
- **Recall:** "Track depth; keep a bracket only when depth>0 (inside)."
- **Algo:** `(` : append if `level>0`, then `level++`. `)` : `level--`, then append if `level>0`.
- **Trap:** Increment order matters — for `(` after the check, for `)` before it.
- **O(N) / O(N).**
- **Pattern:** Depth counter.

P02 — Reverse Words in a String

- **Problem:** Reverse word order, trim & single-space. `" a b "` → `"b a"`.
- **Recall:** "Scan from right: skip spaces → grab word → append with single space."
- **Algo:** From the end, skip spaces, mark `end`, walk over word, `substr(i+1, end-i)`; separator only if result non-empty.
- **Trap:** Conditional separator prevents leading/trailing spaces; `substr` length is `end-i`.
- **O(N) / O(N).**
- **Pattern:** Right-to-left word parse.

P03 — Largest Odd Number in String

- **Problem:** Largest odd-valued prefix (trim from right). `"504"` → `"5"`; `"4206"` → `""`.
- **Recall:** "Odd ⟺ last digit odd → find rightmost odd digit, strip leading zeros."
- **Algo:** Scan right→left for first odd digit `ind`; skip leading zeros; return `s[i..ind]`.
- **Trap:** Scan from the **right** (not left); strip leading zeros; all-even → `""`.
- **O(N) / O(1).**
- **Pattern:** Greedy on digits.

P04 — Longest Common Prefix

- **Problem:** Longest prefix common to all strings. `["interview","internet",...]` → `"inter"`.
- **Recall:** "Sort; LCP of first & last sorted strings = LCP of all."
- **Algo:** Sort; compare `first` vs `last` up to `min` length; stop on mismatch.
- **Trap:** Bound by the shorter of first/last; empty-array guard.
- **O(N·M·log N) / O(1).** (Vertical scan: O(N·M).)
- **Pattern:** Sort + compare extremes.

P05 — Isomorphic Strings

- **Problem:** Consistent one-to-one char mapping s↔t. `"paper"`, `"title"` → true.
- **Recall:** "Last-seen index+1 of each char must agree in both strings."
- **Algo:** `m1[s[i]] != m2[t[i]]` → false; else store `i+1` in both.
- **Trap:** Check **both** directions (rules out two→one); `index+1` so 0 = unseen.
- **O(N) / O(1).**
- **Pattern:** Paired fingerprint / two maps.

P06 — Rotate String

- **Problem:** Is `goal` a rotation of `s`? `"abcde"`, `"cdeab"` → true.
- **Recall:** "Rotation ⟺ same length AND `goal` ⊂ `s+s`."
- **Algo:** Length check; `(s+s).find(goal) != npos`.
- **Trap:** Don't skip the length check → false positives.
- **O(N²) naive find / O(N) with KMP · O(N) space.**
- **Pattern:** Doubling trick.

P07 — Check Anagram

- **Problem:** Is `t` a rearrangement of `s`? `"INTEGER"`, `"TEGERNI"` → true.
- **Recall:** "Same length; `freq` ++ for s, -- for t; all zero ⇒ anagram."
- **Algo:** One 26-array; increment/decrement; verify all zeros.
- **Trap:** Case offset (`'A'` vs `'a'`) / use 256 for general input; length early-exit.
- **O(N) / O(1).**
- **Pattern:** Frequency count.

P08 — Sort Characters by Frequency

- **Problem:** Chars by decreasing frequency (this version: distinct chars). `"tree"` → `['e','r','t']`.
- **Recall:** "Count (freq,char) pairs; sort freq↓ char↑; emit nonzero."
- **Algo:** 26 pairs `(count, char)`; custom comparator; collect nonzero.

- **Trap:** Store char in the pair (survives sort); LC #451 repeats each char `count` times.
- **O(N) / O(1).**
- **Pattern:** Count + sort pairs.

P09 — Max Nesting Depth of Parentheses

- **Problem:** Max parenthesis depth in a valid expr. `"(1+(2*3)+((8)/4))+1"` → 3.
- **Recall:** "`(` depth++, `)` depth--; answer = max depth."
- **Algo:** Counter, `ans = max(ans, depth)` each step; ignore non-brackets.
- **Trap:** No stack needed; update the max after incrementing.
- **O(N) / O(1).**
- **Pattern:** Running counter.

P10 — Roman to Integer

- **Problem:** Roman numeral → int. `"MCMXCIV"` → 1994.
- **Recall:** "value(s[i]) < value(s[i+1]) → subtract, else add; add the last."
- **Algo:** Map symbols; loop to `n-1` comparing with next; add `s.back()`.
- **Trap:** Subtractive pairs; add the final symbol; empty-string `size()-1` underflow.
- **O(N) / O(1).**
- **Pattern:** Compare-with-next simulation.

P11 — String to Integer (atoi)

- **Problem:** Parse int: spaces → sign → digits, clamp to 32-bit. `" -12345"` → −12345.
- **Recall:** "Phases: spaces, sign, digits (long long), clamp to [INT_MIN, INT_MAX]."
- **Algo:** Skip spaces, read sign, accumulate digits; clamp as soon as bound crossed; stop at non-digit.
- **Trap:** Overflow before clamp (use `long long`); sign must precede digits; strict phase order.
- **O(N) / O(N) rec (O(1) iterative).**
- **Pattern:** Phased parse + overflow guard.

P12 — Count Substrings (Exactly K Distinct)

- **Problem:** Substrings with exactly `k` distinct chars. `"pqpqs"`, k=2 → 7.
- **Recall:** "exactly(k) = atMost(k) − atMost(k−1)."
- **Algo:** Sliding window `atMost`: shrink while distinct > k; add `right−left+1`.
- **Trap:** Erase zero-count chars so `freq.size()` = distinct; window contributes `right−left+1`.
- **O(N) / O(k).**
- **Pattern:** Sliding window + atMost trick.

P13 — Longest Palindromic Substring

- **Problem:** Longest palindromic substring. `"babad"` → `"bab"`.
- **Recall:** "Expand around each center — odd `(i,i)` and even `(i,i+1)`."
- **Algo:** For each center, expand while sides match; track longest; result `substr(left+1, right-left-1)`.
- **Trap:** Must try **both** odd and even centers; final substring bounds off-by-one.
- **$O(N^2)$ / O(1).** (Manacher O(N) rarely needed.)
- **Pattern:** Expand around center.

P14 — Sum of Beauty of All Substrings

- **Problem:** Σ (maxFreq − minFreq) over all substrings. `"xyx"` → 1.
- **Recall:** "Fix start i, extend j, maintain freq; add max−min each step."
- **Algo:** Outer `i`, inner `j` increments `freq[s[j]]`; scan ≤26 freqs for min/max; add difference.
- **Trap:** Reuse the freq map (don't rebuild → $O(N^3)$); min over **present** chars only.
- **$O(N^2 \cdot 26)$ / O(26).**
- **Pattern:** Incremental frequency over substrings.

P15 — Reverse Every Word (= P02)

- **Problem:** Reverse word order (same as P02). `" a b "` → `"b a"`.
- **Recall:** "Right-to-left word extraction, single-space join. Identical to P02."
- **Algo:** See P02.
- **Trap:** Same as P02 (conditional separator, substr length).
- **O(N) / O(N).**
- **Pattern:** Right-to-left word parse.

Decision Guide — "Which string technique is this?"

Clue in the problem	Go to
"brackets / parentheses depth or primitives"	P01, P09
"reverse word order / normalize spaces"	P02, P15
"largest/smallest number from digits"	P03
"common prefix"	P04
"consistent character mapping / pattern"	P05
"rotation of a string"	P06
"anagram / same characters"	P07
"sort / rank by frequency"	P08, P14
"roman / parse a number from text"	P10, P11
"substrings with a distinct/count constraint"	P12
"palindrome substring"	P13

Golden Rules

1. **Fixed alphabet → array, general chars → hash map.** 26/256 arrays give O(1) space.
2. **Valid brackets need only a counter**, not a stack.
3. **Parse in ordered phases** (atoi, roman) and clamp overflow with a wider type.
4. **"Exactly k" = atMost(k) − atMost(k−1)** for substring/subarray counting.
5. **Palindromes → expand around all `2n-1` centers** (odd + even).
6. **Word problems → scan from the right**, capture bounds, join with a conditional single space.

Step 5 — Strings (Basic & Medium) · 15 problems · one sheet. Good luck.